

Product Requirements Document (PRD)

AI-Powered Insurance Claim Processing System

This PRD defines the requirements for building an AI-powered Insurance Claim Processing Workflow using LangGraph, Large Language Models (LLMs), and workflow orchestration principles. The objective is to automate insurance claim validation, fraud detection, severity estimation, routing, and customer communication.

1. Problem Statement

Insurance companies receive thousands of claims daily across multiple categories such as: Vehicle accidents Medical reimbursements Property damage Theft claims Manual processing creates several challenges: Slow approval cycles High operational costs Fraudulent claims slipping through Poor customer experience Inconsistent decision-making The goal is to build an AI workflow system capable of automating major stages of claim processing while supporting human review for high-risk cases.

2. Objectives

Automate insurance claim analysis Detect incomplete or suspicious claims Reduce claim processing time Improve fraud detection accuracy Provide dynamic workflow routing using LangGraph Generate professional customer responses automatically

3. Scope

In Scope: Claim extraction from text/PDF input Fraud risk detection Conditional workflow routing Severity estimation Approval/rejection workflow Human-in-the-loop escalation Automated communication generation **Out of Scope:** Real payment disbursement Production database deployment External API integrations Real OCR/image processing pipelines

4. Functional Requirements

FR-1: Claim Information Extraction Extract customer name Extract claim type Extract incident summary Extract estimated repair cost Identify attached documents **FR-2: Claim Completeness Validation** Validate mandatory fields Detect missing documents Classify claim as COMPLETE or INCOMPLETE **FR-3: Fraud Risk Analysis** Identify suspicious wording Analyze unusually high claim amounts Flag repeated incident patterns Assign risk score **FR-4: Damage Severity Estimation** Classify severity into Minor, Moderate, Severe, or Total Loss **FR-5: Dynamic Approval Routing** Auto-approve low-risk claims Escalate medium-risk claims for manual review Send high-risk claims to fraud investigation **FR-6: Automated Communication** Generate approval emails Generate rejection emails Generate document request emails Generate investigation notices

5. Non-Functional Requirements

System should support modular node-based architecture Workflow must support conditional routing System should provide execution logs Nodes should be independently testable Workflow should support retries and error handling Responses must be professional and safe

6. Workflow Architecture

Expected Workflow: START → Claim Extraction → Completeness Validation If INCOMPLETE: → Request Documents → END If COMPLETE: → Fraud Analysis → Severity Estimation → Approval Decision Routes: Low Risk → Auto Approval Medium Risk → Manual Review High Risk → Fraud Investigation

7. Technical Requirements

Component Technology Workflow Engine LangGraph LLM Framework LangChain Language Model OpenAI / Claude / Groq Validation Pydantic Monitoring LangSmith UI (Optional) Streamlit

8. Deliverables

Students must submit the following: GitHub Repository Workflow Diagram README Documentation Graph Visualization Demo Video Sample Inputs and Outputs Error Handling Demonstration

9. Evaluation Criteria

Criteria Marks LangGraph Workflow Design 20 Routing Logic 15 Fraud Detection Design 15 Structured Outputs 10 Error Handling 10 Code Quality 10 State Management 10 Innovation 10

10. Bonus Features

OCR support for uploaded PDFs Multilingual claim handling Human-in-the-loop approvals Slack/email notifications Checkpointing and memory Multi-agent architecture Historical claim similarity search

11. Sample Input

Customer Name: John Doe
Claim Type: Car Accident
Description: A truck hit my parked vehicle during heavy rain.
Estimated Repair Cost: \$4,500
Police Report: Attached
Photos: Attached

12. Suggested Folder Structure

```
insurance_claim_ai/  
■■■ main.py  
■■■ graph.py  
■■■ state.py  
■■■ nodes/  
■ ■■■ extractor.py  
■ ■■■ validator.py  
■ ■■■ fraud.py  
■ ■■■ severity.py  
■ ■■■ approval.py  
■ ■■■ response_generator.py
```

```
■■■ prompts/  
■■■ utils/  
■■■ README.md
```